
UNIDAD 07

Arrays unidimensionales

Programación en Computación · Ingeniería Electromecánica — 2.º año · UTN FR Reconquista
Longhi Pablo · Talijancic Iván

Qué vamos a ver

01 **El problema** que resuelven los vectores

02 Declarar, inicializar y acceder por **índice**

03 **Recorrer** con `for` y con `while`

04 Los **cinco algoritmos** fundamentales

05 `.slice()` : la trampa de las **referencias**

Requisitos: unidad 05 (bucles) y unidad 06 (funciones). Todo lo de hoy se escribe dentro de funciones.



PARTE 1

| El problema

Tres mediciones de tensión

app.js

```
let medicion1 = 219.4  
let medicion2 = 221.8  
let medicion3 = 218.2
```

Funciona perfecto. Tres valores, tres variables.

Ahora las de todo el mes

app.js

```
let medicion1 = 219.4
let medicion2 = 221.8
let medicion3 = 218.2
// ... 27 líneas más ...
let medicion31 = 220.1
```

Inviabile. Y el problema real todavía no apareció.

PREGUNTA

¿Y si el usuario decide cuántas mediciones son?

No podés declarar variables que todavía no sabés cuántas son.



Un nombre, muchos valores

app.js

```
let mediciones = [219.4, 221.8, 218.2]
```

MEDICIONES

219.4	221.8	218.2
0	1	2

Todos los elementos comparten el **nombre**. Los distingue el **índice**.

PARTE 2

| Declarar y acceder

Tres formas de declarar

Vacío

```
let v = []
```

Cuando no sabés la dimensión todavía.

Con valores

```
let v = [1, 2, 3]
```

Cuando los datos son conocidos.

Con dimensión

```
let v = new Array(5)
```

5 posiciones, todas sin inicializar.

En esta cátedra los vectores se inicializan **siempre con un bucle**. No usamos `fill()` ni `push()`.

Leer y modificar

app.js

```
let nombres = ['Juan', 'Ana', 'Luis']  
  
console.log(nombres[0])  
console.log(nombres[2])  
  
nombres[1] = 'María'  
console.log(nombres)
```

SALIDA

```
Juan  
Luis  
[ 'Juan', 'María', 'Luis' ]
```

.length: cuántos elementos tiene

app.js

```
let v = ['Juan', 'Ana', 'Luis']  
console.log(v.length) // 3
```

3 ELEMENTOS • ÚLTIMO ÍNDICE 2

'Juan'	'Ana'	'Luis'
0	1	2

CONCEPTO

VALOR

Cantidad de elementos

v.length

Índice del primero

0

Índice del último

v.length - 1

PREGUNTA

¿Qué imprime `nombres[3]`?

```
let nombres = ['Juan', 'Ana', 'Luis']  
console.log(nombres[3])
```



undefined, y sin avisar

app.js

```
let nombres = ['Juan', 'Ana', 'Luis']  
  
console.log(nombres[3])  
console.log(nombres[3] + 1)
```

SALIDA

```
undefined  
NaN
```

ERROR FRECUENTE

JavaScript **no tira error** al salirse de rango. El programa sigue con basura adentro y el error explota mucho más adelante, lejos de donde lo causaste. **El control del rango es tu responsabilidad.**

PARTE 3

| Recorrer el vector

Con `for`

app.js

```
const imprimir = (v) => {  
  for (let i = 0; i < v.length; i++) {  
    console.log(`${i}: ${v[i]}`)  
  }  
}  
  
imprimir([10, 20, 30])
```

SALIDA

```
0: 10  
1: 20  
2: 30
```

Es la forma más común: sabés exactamente cuántas iteraciones necesitás, tantas como elementos tenga el vector.

< 0 <=

Para un vector de **5 elementos**:

CONDICIÓN	ÚLTIMO I	ACCEDA A	
<code>i < v.length</code>	4	índices 0, 1, 2, 3, 4	✓
<code>i <= v.length</code>	5	<code>v[5]</code> → <code>undefined</code>	✗

ERROR FRECUENTE

Cinco elementos, **último índice 4**. Es el error número uno al empezar con vectores.

Con `while`

app.js

```
let i = 0

while (i < v.length) {
  console.log(v[i])
  i++
}
```

01 **Inicializar** el contador antes del bucle

02 **Condición** de corte

03 **Incrementar** dentro del bucle

ERROR FRECUENTE

Si te olvidás el `i++`, la condición nunca se hace falsa: **bucle infinito**. Se corta con `Ctrl + C` en la terminal.

¿Cuál uso?

for

Recorrer el vector completo.
Sabés cuántas iteraciones son.

while

Cortar antes de llegar al final.
La cantidad depende de una condición.

`for` por defecto. `while` cuando necesitás cortar.

PARTE 4

| Los cinco algoritmos

Acumular

app.js

```
const sumar = (v) => {  
  let suma = 0  
  
  for (let i = 0; i < v.length; i++) {  
    suma += v[i]  
  }  
  
  return suma  
}
```

TRAZA CON [5, 10, 15]

I	V[I]	SUMA
—	—	0
0	5	5
1	10	15
2	15	30

El acumulador arranca en 0 y se declara **fuera** del bucle. Adentro, se reinicia en cada vuelta.

Contar según una condición

app.js

```
const contarPares = (v) => {  
  let cantidad = 0  
  
  for (let i = 0; i < v.length; i++) {  
    if (v[i] % 2 === 0) {  
      cantidad++  
    }  
  }  
  
  return cantidad  
}
```

Igual que acumular, pero el contador sube **de a uno** y sólo cuando se cumple el `if`.

Máximo y su posición

app.js

```
const buscarMaximo = (v) => {  
  let maximo = v[0] // el PRIMERO, no 0  
  let posicion = 0  
  
  for (let i = 1; i < v.length; i++) { // arranca en 1  
    if (v[i] > maximo) {  
      maximo = v[i]  
      posicion = i  
    }  
  }  
  
  return [maximo, posicion] // dos datos → un vector  
}
```

PREGUNTA

¿Por qué no `let maximo = 0`?

```
buscarMaximo([-5, -12, -3, -40])
```



Con temperaturas bajo cero

VECTOR DE TRABAJO

-5	-12	-3	-40
0	1	2	3

INICIALIZANDO EN

```
let maximo = 0
```

```
let maximo = v[0]
```

RESULTADO

0 — un valor que no está en el vector ❌

-3 ✅

ERROR FRECUENTE

En los parciales esto se corrige como **error grave**: denota no entender qué representa el acumulador.

Búsqueda lineal

app.js

```
const buscarPosicion = (v, buscado) => {  
  let posicion = -1  
  let i = 0  
  
  while (i < v.length && posicion === -1) { // corta al encontrarlo  
    if (v[i] === buscado) posicion = i  
    i++  
  }  
  
  return posicion // -1 si no está  
}
```

-1 es un índice **imposible**: así distinguís «no está» de «está en la posición 0».

Vector aleatorio

app.js

```
const rndInt = (min, max) =>
  Math.floor(Math.random() * (max - min + 1)) + min

const generarVector = (dim, min, max) => {
  const v = new Array(dim)

  for (let i = 0; i < dim; i++) {
    v[i] = rndInt(min, max)
  }

  return v.slice()
}
```

`rndInt()` ya la escribimos en la unidad 06. Se reusa de acá en adelante.

PARTE 5

| La trampa de `.slice()`

Un número se pasa por copia

app.js

```
const duplicar = (n) => {  
  n = n * 2  
  return n  
}  
  
let x = 5  
const y = duplicar(x)
```

SALIDA

```
x = 5      ← intacto  
y = 10
```

La función recibe una **copia** del valor. El original no se toca.

Un vector, no

app.js

```
const duplicarMal = (v) => {  
  for (let i = 0; i < v.length; i++) {  
    v[i] = v[i] * 2  
  }  
  return v  
}  
  
const original = [1, 2, 3]  
const duplicado = duplicarMal(original)
```

PREGUNTA

¿Cuánto vale **original** ahora?

```
const original = [1, 2, 3]
const duplicado = duplicarMal(original)

console.log(original)
```



Se arruinó el original

SALIDA

```
Original: [ 2, 4, 6 ]    ← se arruinó  
Duplicado: [ 2, 4, 6 ]
```

Número

Se pasa por **copia**. La función trabaja sobre un valor propio.

Vector

Se pasa por **referencia**. Apunta al mismo lugar en memoria.

La solución

× ASÍ NO

```
const dup = (v) => {  
  for (...) {  
    v[i] = v[i] * 2  
  }  
  return v  
}
```

✓ ASÍ SÍ

```
const dup = (v) => {  
  const r = v.slice()  
  for (...) {  
    r[i] = r[i] * 2  
  }  
  return r  
}
```

Usá `.slice()` cuando una función **recibe** un vector que no debe modificar, y cuando **retorna** uno.

Los errores que vamos a cometer

ERROR	SÍNTOMA
<code>i <= v.length</code>	<code>undefined</code> o <code>NaN</code> al final
Acumulador dentro del bucle	Sólo queda el último elemento
<code>let maximo = 0</code>	Falla con valores negativos
Olvidar <code>i++</code> en un <code>while</code>	El programa se cuelga
Olvidar <code>parseInt()</code>	La suma concatena: <code>'53'</code>
Modificar el vector recibido	Se corrompen los datos de entrada

Reglas de la cátedra

Podés usar

`for` · `while` · `do-while` · `if` · `switch` · acceso por índice
· `.length` · `.slice()`

No podés usar

`map` · `filter` · `reduce` · `forEach` · `find` · `sort` · `flat` ·
`indexOf` · `splice` · `fill()` · `push()` · retornar objetos

Primero dominás la lógica del recorrido.

LAS HERRAMIENTAS QUE LA RESUELVEN POR VOS VIENEN DESPUÉS

TRABAJO PRÁCTICO

Análisis de mediciones de tensión de línea

7 problemas · todo modularizado en funciones · `tp.md`

Apunte completo de la unidad: `apunte.md`